



docker logs Command

Introduction

The **docker logs** command is used to **view the logs of a running or stopped container**.

- What is **docker logs** used for?
- How does it help in debugging and monitoring?
- How can you use different options with **docker logs**?

This guide explains **how to use docker logs to track container events and troubleshoot issues**.

Section 1: Learn

What is **docker logs**?

docker logs allows you to **see the standard output (stdout) and standard error (stderr) logs** generated by a container.

Feature	Benefit
Tracks container output	Helps debug application issues
Shows real-time logs	Monitors live activity in containers
Works on running & stopped containers	Useful for post-mortem debugging
Supports filtering & formatting	Helps extract specific log information



How **docker logs** Works

- When an application runs inside a container, it **prints messages to stdout or stderr**.
- **docker logs** fetches and **displays these messages** in the terminal.

Interesting Fact

Logs are stored **inside the container's filesystem**, meaning **if the container is removed, logs are lost unless saved externally**.

Section 2: Practice

1. Viewing Logs of a Running Container

```
docker logs <container_id>
```

Example Output:

```
Starting Nginx server...  
Listening on port 80...  
GET /index.html 200 OK
```

Why is this useful?

- Shows real-time events happening inside the container.
-

2. Viewing Logs with Timestamp

```
docker logs --timestamps <container_id>
```

Example Output:



```
2024-02-26T10:00:00Z Starting Nginx server...
```

```
2024-02-26T10:00:05Z Listening on port 80...
```

Why is this useful?

- Helps **track the exact time** of each event.
-

3. Viewing Only Recent Logs

To see **only the last 10 lines** of logs:

```
docker logs --tail 10 <container_id>
```

To continuously monitor logs **as new ones are generated**:

```
docker logs -f <container_id>
```

This works like **Linux tail -f**, showing real-time updates.

4. Filtering Logs Based on Time

To see logs **only from the last 30 minutes**:

```
docker logs --since 30m <container_id>
```

To see logs **from a specific time**:

```
docker logs --since "2024-02-26T10:00:00Z" <container_id>
```

Why is this useful?

- Helps **debug specific time periods**.
-

5. Saving Logs to a File

Redirect logs to a file for further analysis:



```
docker logs <container_id> > container_logs.txt
```

Why is this useful?

- Useful for **sharing logs** with a team.
-

Section 3: Know More

Frequently Asked Questions

1. **How do I check logs of a stopped container?**
 - Logs are still accessible using `docker logs <container_id>`.
2. **Can I clear logs of a running container?**
 - No, but you can restart the container:

```
docker restart <container_id>
```

3. **How do I check logs of all containers?**
 - Use:

```
docker ps -a --format "{{.ID}}" | xargs -n1 docker logs --tail 10
```

4. **Can I view logs in JSON format?**
 - Yes, if the logging driver supports it:

```
docker logs --format json <container_id>
```

5. **How do I change Docker's log storage location?**
 - Configure `log-driver` in Docker's `daemon.json` file.
-

Conclusion

The `docker logs` command **helps monitor, debug, and analyze container behavior**.



Start by **checking container logs**, **filtering by time**, and **saving logs for future analysis**, and soon you'll be **troubleshooting Docker efficiently**!